Reviewer Pack — Minimal Brauer Group Rank Correlation with Circuit Monotone …

Entry 2cb1217ab7fd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 12:59:29 UTC

Minimal Brauer Group Rank Correlation with Circuit Monotone Width

Entry ID: `2cb1217ab7fd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 12:59:29 UTC

1. Conjecture

Field A (mathematical branch): Brauer Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every Boolean circuit C with n variables, the minimal rank of its associated Brauer group B(C) is linearly correlated with its monotone width w_mon(C), such that |B(C)| = Θ(w_mon(C)).

Rationale (proposer's reasoning):

> The Brauer group provides a classification of algebraic structures related to field extensions and Galois theory. If this invariant is correlated with the circuit's monotone width, it could suggest an underlying structure that influences computational complexity.

Taxonomy category: `BrauerGroupMonotoneWidth` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `899e7255b9a650f0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the slope of the linear regression between |B(C)| and w_mon(C) is within ±10% of 1, with p-value ≤ 0.05 for at least 80% of seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.90	UNCERTAIN	SAFE
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot row
        max_row = i
        for r in range(i+1, rows):
            if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                max_row = r
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate non-pivot elements
        pivot = matrix[i][i]
        for j in range(i, cols):
            matrix[i][j] /= pivot
        for r in range(rows):
            if r != i:
                factor = matrix[r][i]
                for j in range(i, cols):
                    matrix[r][j] -= factor * matrix[i][j]

    rank = sum(1 for row in matrix if any(row))
    return rank

def compute_brauer_group(circuit):
    # Convert circuit to a string representation
    circuit_str = ' OR '.join(str(gate) for gate in circuit)

    # Simulate Brauer group computation (constructive mapping)
    # For simplicity, assume each gate contributes a 2x2 matrix modulo 2
    matrices = [[1, 0], [0, 1]] * len(circuit)
    result_matrix = []
    for mat in matrices:
        if not result_matrix:
            result_matrix = mat
        else:
            # Matrix multiplication modulo 2
```

```
            new_row = []
            for i in range(len(mat)):
                sum_val = 0
                for j in range(len(mat[0])):
                    sum_val += (mat[i][j] * result_matrix[j]) % 2
                new_row.append(sum_val)
            result_matrix = [new_row]

    return gaussian_elimination(result_matrix)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 30
    ranks = []
    widths = []

    for _ in range(instances_tested):
        n = random.randint(5, n_max)
        circuit = [random.choice([0, 1]) for _ in range(n)]

        rank = compute_brauer_group(circuit)
        width = sum(circuit)  # Monotone width is the number of gates

        ranks.append(rank)
        widths.append(width)

    if not ranks or not widths:
        return {
            "metric_name": "Brauer Group Rank",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "empty_ranks_or_widths"
        }

    # Perform linear regression
    mean_rank = sum(ranks) / len(ranks)
    mean_width = sum(widths) / len(widths)
    ss_tot = sum((w - mean_width)**2 for w in widths)
    ss_res = sum((r - (mean_rank * w / mean_width))**2 for r, w in zip(ranks, widths))

    if ss_tot == 0:
        return {
            "metric_name": "Brauer Group Rank",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "constant_width"
        }

    r_squared = 1 - (ss_res / ss_tot)
    slope = mean_rank * (mean_width / len(widths))
```

```
    return {
        "metric_name": "Brauer Group Rank",
        "metric_value": slope,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": abs(slope - 1) <= 0.1 and r_squared >= 0.95,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 7 for i in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slope = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_slope = math.sqrt(sum((r["metric_value"] - mean_slope)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample='slope_outside_range' first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41058d21.py", line 125, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41058d21.py", line 75, in run_trial
    rank = compute_brauer_group(circuit)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41058d21.py", line 57, in compute_braue
    for j in range(len(mat[0])):
                   ^^^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete the computation necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, rerun the test with a valid dataset to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13919
2	preregistration	ollama_remote	glm4:latest	0	0	9247
3	novelty	ollama_remote	glm4:latest	0	0	8432
4	novelty	ollama_remote	glm4:latest	0	0	20869
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14816
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	101209
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34506
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20147
9	judge	ollama_remote	glm4:latest	0	0	8897

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 232041 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2cb1217ab7fd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2cb1217ab7fd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2cb1217ab7fd.tar.gz` (if generated)